

FINDING THE OPTIMAL PATH IN AN ARTIFICIAL ENVIRONMENT USING Q-LEARNING



Artificial Intelligence Foundations
(CIS4049-N)

Atla Rana Dheeraj
(W9640446)

Word Count: 2063

Table of Contents

Abstract:	3
Introduction:	3
Define the Challenges:	3
Methodology Used	5
Breadth First Search (BFS):	5
Depth First search (DFS):	6
Q-learning:	6
Research Methodology:	6
Breadth First Search:	6
Q-learning:	8
Results and Discussions:	10
Conclusion:	13
References:	14

Abstract:

Artificial learning encompasses various machine learning approaches, including supervised, unsupervised, and reinforcement learning, each tailored to different data types. This report focuses on the application of reinforcement learning, particularly in navigating unfamiliar environments and systematically addressing diverse situations through iterative exploration.

Q-learning, a prominent technique within this domain, utilizes a reward-action framework to identify optimal pathways based on accumulated rewards. The study adopts a graph structure where nodes are manually categorized into four distinct classes, emphasizing their significance. The methodology involves strategically avoiding specific nodes and implementing reward adjustments, incorporating penalties and additional rewards to enhance navigation and decision-making in dynamic environments.

Introduction:

In the realm of graph-based navigation, the breadth-first search path length syntax emerges as a reliable tool for effortlessly identifying optimal paths. However, challenges arise when avoiding specific nodes, demanding innovative solutions. Q-learning, a key methodology adept at defining actions and rewards in the face of unknown environments, providing a robust framework for overcoming such intricacies.

This study extends its exploration beyond traditional applications, venturing into an artificial environment manually constructed through a graph. Here, node names undergo transformation to reflect distinct categories: Cyber Police, Target User, Infected User, and Normal User. The ultimate objective unfolds as the identification of a hacker in a final scenario.

To facilitate these investigations, the study employs the NetworkX Python library, leveraging its capabilities for random graph generation. Numeric nodes and strategic layouts are implemented, with the graph's connectivity guided by a probability-driven scheme. This introduction sets the stage for a comprehensive exploration of strategies within dynamic environments, combining the power of graph theory and Q-learning for nuanced problem-solving.

Define the Challenges:

The primary objective of the problem is pathfinding, with a focus on adhering to specified rules and regulations throughout the journey towards the destination. To address this, a random graph comprising 300 nodes was meticulously crafted, incorporating a connection probability of 0.4 between nodes. Notably, the edges of the resulting graph, configured in a dashed pattern, signify a non-directed graph. The inclusion of a seed ensures consistency, guaranteeing the utilization of the same graph each time the kernel is initiated. The visual representation of the generated graph is illustrated in Figure 1.

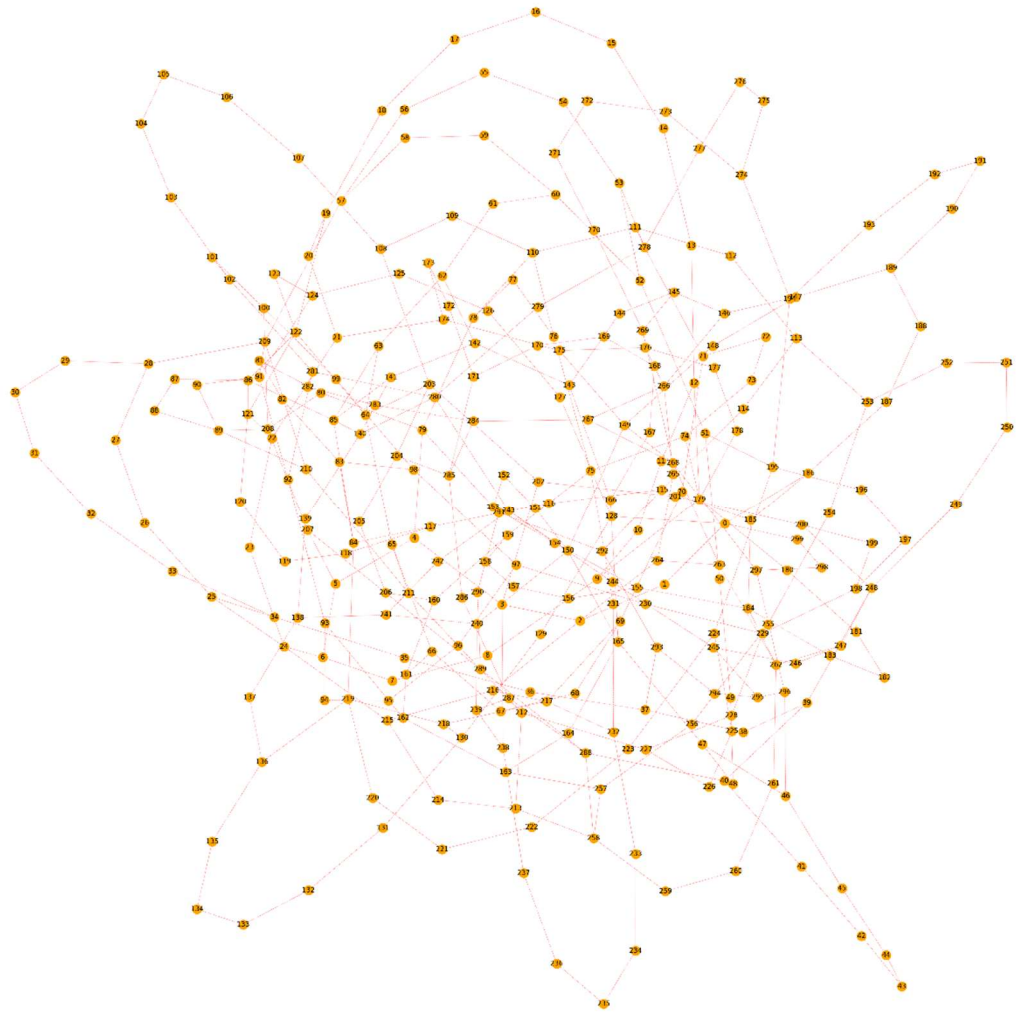


Figure 1: The Random Graph

The manual adaptation of the environment involves a strategic modification of node names and colours within the existing graph. Specifically, node 91 has undergone a transformation, now labelled as "hacker" and distinguished by a vibrant red hue, signifying it as the ultimate target. Nodes 100 to 200 have been reassigned to represent infected devices, characterized by a cool blue coloration. Furthermore, nodes 200 to 220 have been designated for cyber police, each adorned with a distinctive colour. In parallel, nodes spanning from 20 to 80 have been relabelled as target users, aesthetically rendered in a subtle grey shade. The remaining nodes have assumed the identity of normal users, distinguished by a warm yellow tint. The resultant graph capturing these modifications is visually depicted in Figure 2.

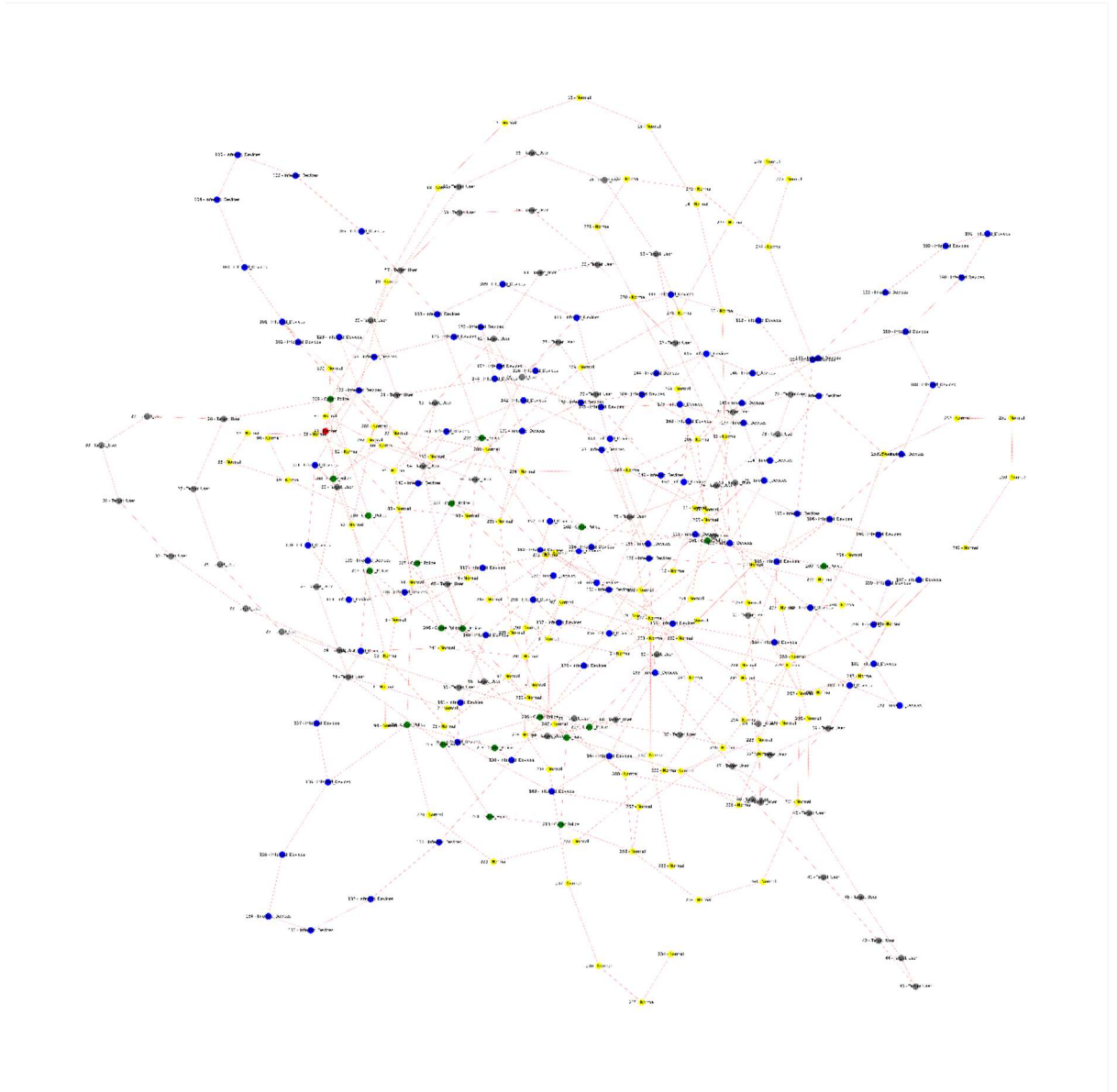


Figure 2: The graph with nodes relabelled.

The guidelines for optimal pathfinding dictate a specific sequence to be observed. Commencing from the initial node, designated as the starting point for the cyber police, the objective is to navigate towards the destination, node 91, identified as the hacker node. In the pursuit of this path, intermediate nodes are to be prioritized based on a predetermined order: Target users hold the foremost position, followed by normal users, cyber nodes, and lastly, infected nodes, which carry a lower priority. This structured prioritization ensures a systematic and efficient traversal through the graph.

Methodology Used

Breadth First Search (BFS):

The breadth-first search (BFS) algorithm systematically explores the graph in a breadthwise manner, contributing to the pathfinding process. In the context of Q-learning, the graph undergoes examination through BFS, yielding a discerned path. The path highlighted in this manner serves as a reference for comparison in the Q-learning methodology, elucidating the trajectory identified through the BFS search approach.

Depth First search (DFS):

The depth-first search (DFS) algorithm delves into the graph in a depth-wise manner, employing a stack operation to traverse through all nodes and explore various depths of the graph. This method entails a thorough and extended exploration along a particular path.

Q-learning:

The Q-learning is a reinforcement learning technique where the agent is designed to learn optimal strategies for decision-making where the action receives the feed back in the form of rewards or penalties.

Formula for finding the next state.

$$Q^{new}(s_t, a_t) \leftarrow \underbrace{Q(s_t, a_t)}_{\text{old value}} + \underbrace{\alpha}_{\text{learning rate}} \cdot \underbrace{\left(\underbrace{r_t}_{\text{reward}} + \underbrace{\gamma}_{\text{discount factor}} \cdot \underbrace{\max_a Q(s_{t+1}, a)}_{\text{estimate of optimal future value}} - \underbrace{Q(s_t, a_t)}_{\text{old value}} \right)}_{\text{new value (temporal difference target)}}$$

State (s): Represents a specific situation or configuration in the environment where the agent finds itself.

Action (a): Refers to the choices available to the agent at a given state. The agent selects an action based on its current knowledge or policy.

Q-value (Q): Represents the expected cumulative reward of taking a particular action in a specific state. The Q-value is updated iteratively as the agent learns from its interactions with the environment.

Policy (π): The strategy or set of rules that the agent follows to determine its actions in different states. Q-learning helps the agent improve its policy over time.

Reward (r): Instantaneous feedback received by the agent after taking a specific action in a particular state. The agent's goal is to learn a policy that maximizes the total cumulative reward.

α (alpha) is the learning rate, controlling the extent to which new information overrides old information.

Research Methodology:

Breadth First Search:

The queue is used for finding the path where the neighbours of the visited node and finding the shortest path using the below pseudo code.

The pseudo-code of BFS algorithm

```
function BFS(graph, start, goal):
    explored = []
    queue = [[start]]

    if start == goal:
        print("Same Node")
        return

    while queue is not empty:
        path = queue.pop(0)
        node = path[-1]

        if node not in explored:
            neighbors = graph[node]

            for neighbor in neighbors:
                new_path = list(path)
                new_path.append(neighbor)
                queue.append(new_path)

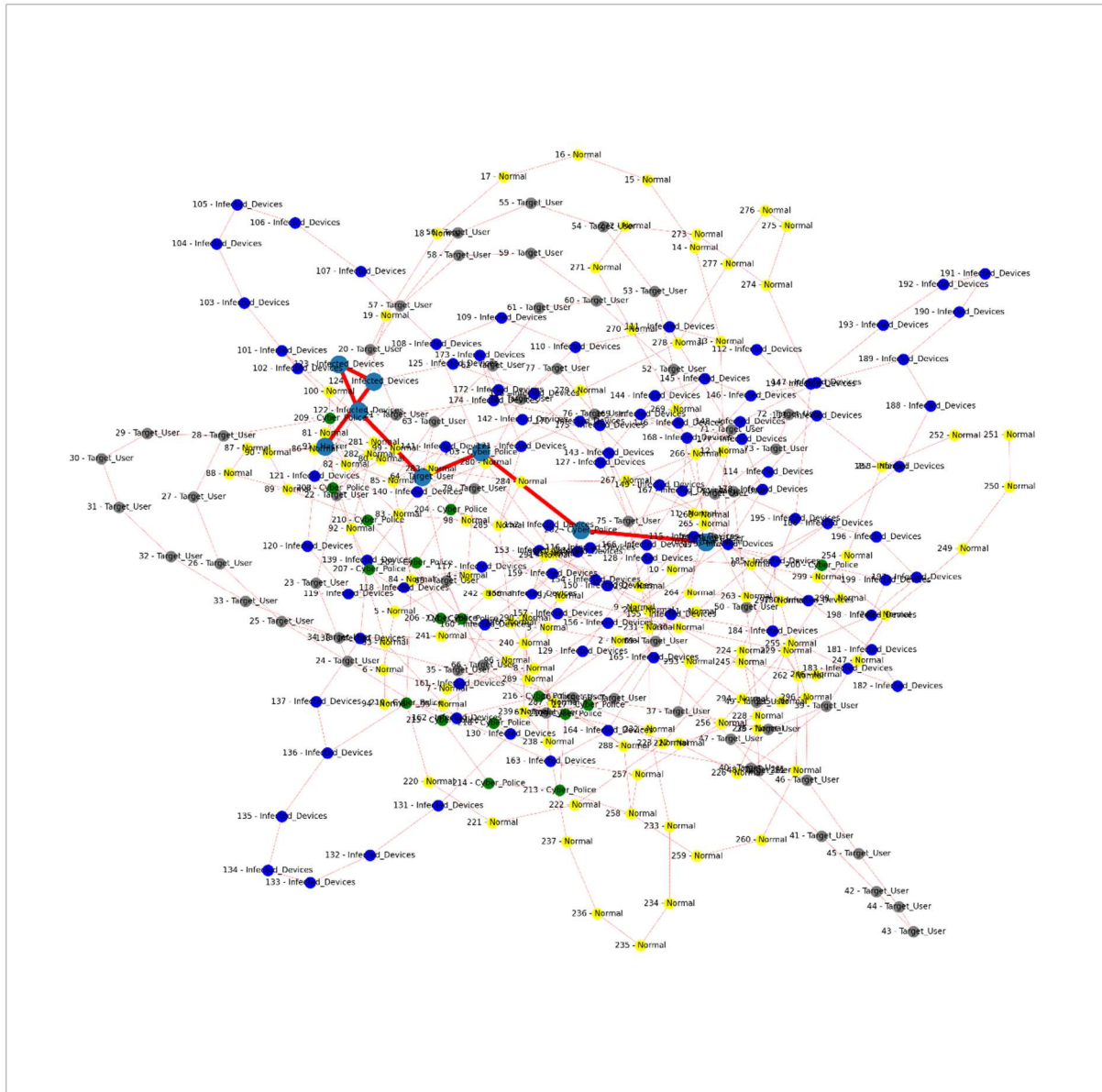
                if neighbor == goal:
                    print("Shortest Path =", *new_path)
                    highlight_path(new_path)
                    return

            explored.append(node)

    print("No Connection")
    return
```

Figure 3: pseudo code of BFS algorithm

The graph of the shortest path using the BFS is highlighted for the path obtained for the graph is in colour red which displays like in the following graph the shortest path.



shortest path= 201 - Cyber_Police 202 - Cyber_Police 203 - Cyber_Police 64 - Target_User 122 - Infected_Devices 123 - Infected_Devices 124 - Infected_Devices 91 - Hacker

Figure 4: Highlighted path of BFS search

Q-learning:

Create a matrix M based on the connection of the graph with value of -1 for no connection and 0 for the connections between the nodes and goal node with 100.

```

[[-1.  0. -1. ... -1. -1.  0.]
 [ 0. -1.  0. ... -1. -1. -1.]
 [-1.  0. -1. ... -1. -1. -1.]
 ...
 [-1. -1. -1. ... -1.  0. -1.]
 [-1. -1. -1. ...  0. -1.  0.]
 [ 0. -1. -1. ... -1.  0. -1.]]

```

Figure 5: Matrix of the connected nodes

Initialize four matrices to represent distinct environments: infected, police, normal, and target, with all values set to zero. Simultaneously, initialize the Q-matrix with zeroes. Subsequently, update the environment matrices by incorporating sample values indicative of police, target, infected, and normal nodes, systematically increasing each count by one for every instance. Introduce a penalty factor of 0.4 specifically for infected nodes. In contrast, specify reward factors of 0.1 for normal nodes and 0.2 for target nodes, without assigning a particular reward factor for infected nodes. This nuanced configuration is crafted to optimize pathfinding, emphasizing the avoidance of infected nodes and prioritizing target nodes. In this strategy, the police environment stands out as the pivotal element steering the Q-learning process.

```

Initialize the Q-table with random values or zeros
Initialize environment matrices for tracking rewards and penalties

reward_factor = # Adjust as needed
penalty_factor = # Adjust as needed

for each episode:
    Initialize state
    repeat until the episode is finished:
        Choose an action using an exploration-exploitation strategy (e.g., epsilon-greedy)
        Take the chosen action and observe the next state and immediate reward
        Collect environmental data (e.g., police presence, infected presence, target presence, and reward presence)

        # Calculate penalties and rewards based on environmental data
        penalty_infected = penalty_factor * env_infected[state, action]
        penalty_normal = penalty_factor * env_normal[state, action]
        reward_target = reward_factor * env_target[state, action]

        # Update the Q-value for the current state-action pair using the Q-learning update rule
        Q(state, action) = Q(state, action) + alpha * (reward + gamma * max(Q(next_state, all_actions)) - Q(state, action) - penalty_infected - penalty_normal + reward_target)

        # Update environmental matrices based on the current action
        update_environment_matrices(state, action, env_infected, env_normal, env_target)

    Move to the next State
    end repeat
end for

function update_environment_matrices(state, action, env_infected, env_normal, env_target):
    # Update environmental matrices based on the current action
    # Adjust as needed based on the specifics of your environment
    # Example: incrementing counters in the matrices based on police presence, drug traces, etc.
    # ...
end function

```

Figure 6: the pseudo code for Q-learning

The optimized path, which strategically avoids infected nodes while prioritizing the acceptance of target nodes, is depicted in Figure 7 of the resulting graph.

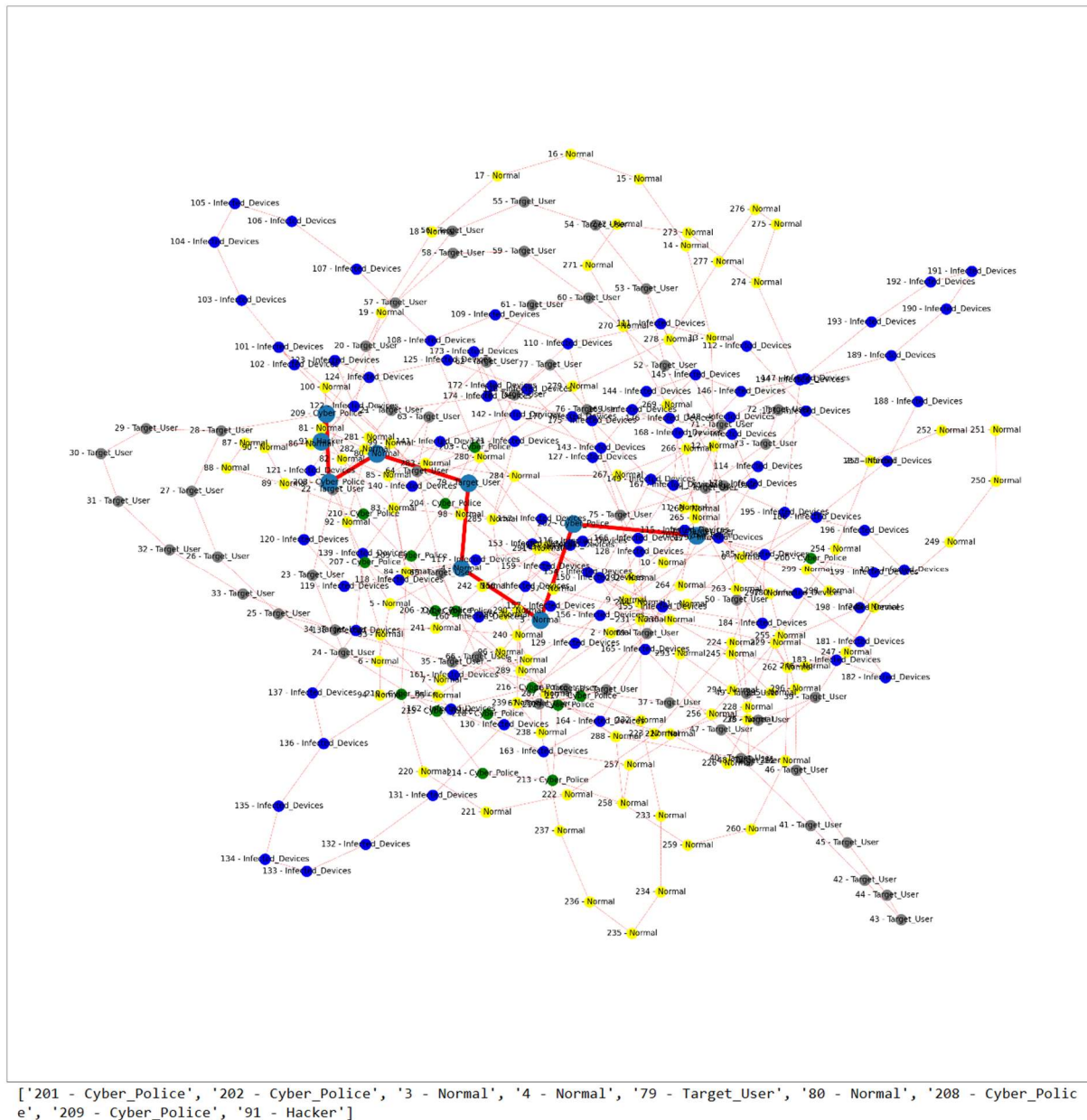


Figure 7: Q-learning path

Results and Discussions:

The graph nodes are assigned a probability of 4/100, ensuring that each node is connected to at least three other nodes. However, the BFS code, aimed at finding the shortest path between these nodes, appears simplistic as it deviates from the specified rules in the example. This deviation hinders the Q-learning process, and modifying the iteration range for the sample produces varying paths.

Index	Optimal Route Method	Implications and number of Iterations	Route	Path Length

1	Breadth First Search	NO	201 – Cyber Police 202 – Cyber Police 203 – Cyber Police 64 – Target User 122 – Infected Devices 123 – Infected Devices 124 – Infected Devices 91 - Hacker	8 stages
2	Depth First Search	No	enlarged	176 stages
3	Q-learning	Yes and 10,000 iterations	201 – Cyber Police', '202 – Cyber Police', '203 – Cyber Police', '64 – Target User', '122 – Infected Devices', '123 – Infected Devices', '124 – Infected Devices', '91 - Hacker'	8 stages
4	Q-learning	Yes and 50,000 iterations	'201 – Cyber Police', '202 – Cyber Police', '3 - Normal', '4 - Normal', '79 – Target User', '80 - Normal', '208 – Cyber Police', '209 – Cyber Police', '91 - Hacker'	9 stages

Adjusting the number of iterations from 10,000 to 100,000 brings about a discernible transformation in the rewards within the graph. Initially resembling the graph portraying the shortest path, the dynamics evolve distinctly. Figures 8 and 9 visually encapsulate this progression, vividly illustrating the impact of increased iterations on the evolving path structure.

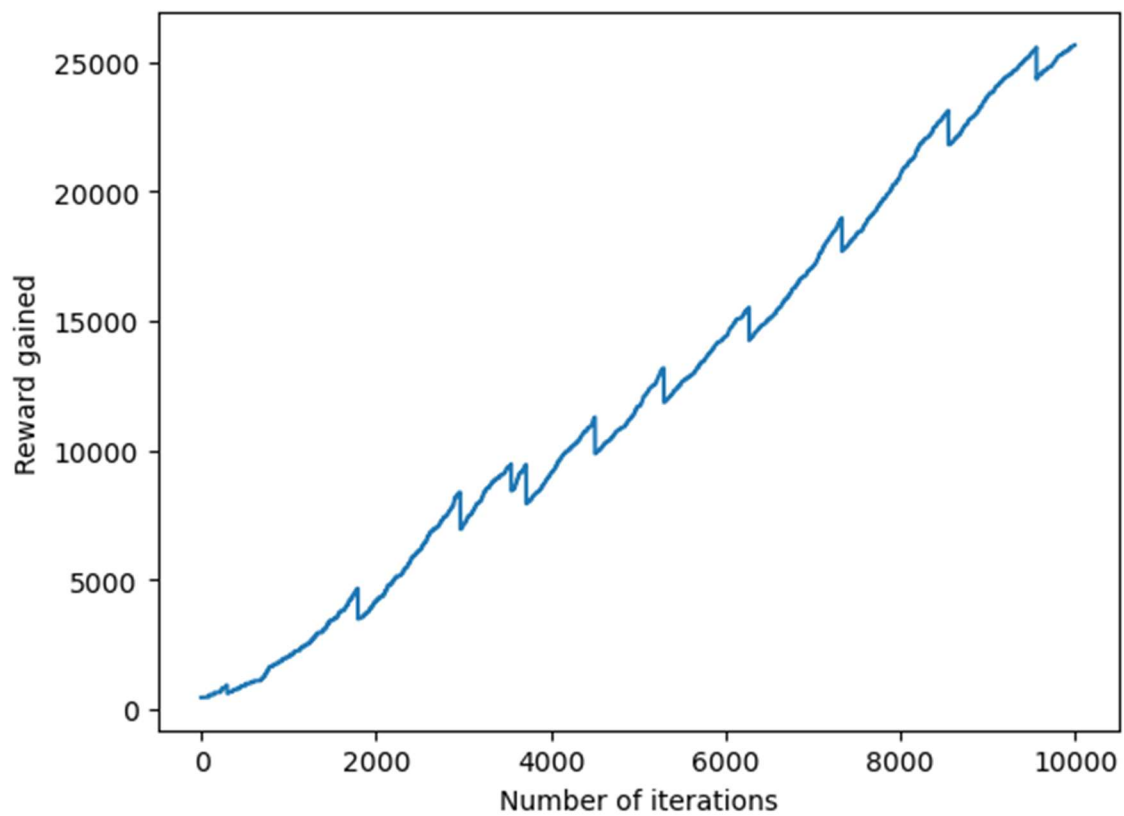


Figure 8: Reward gained for 10,000 iterations.

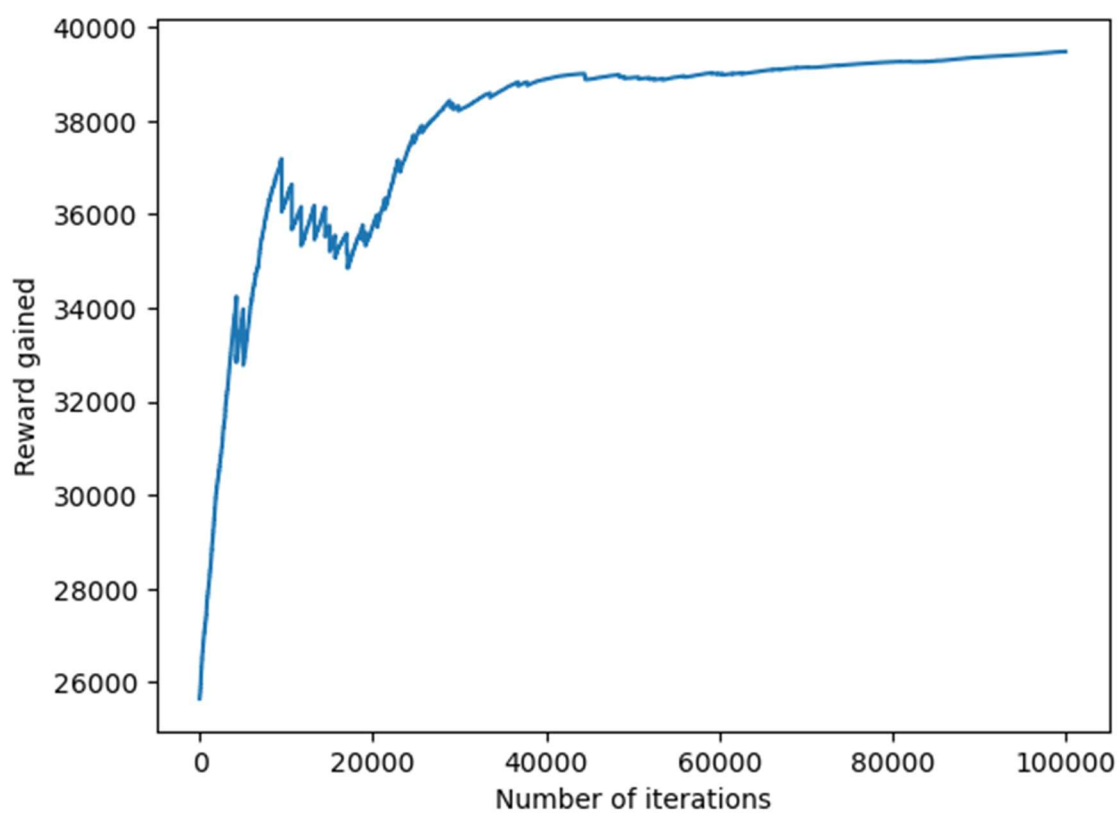


Figure 9: Reward gained after 100,000 iterations.

The table obtained by Dijkstra shortest path length and Q-learning for random nodes.

Start state	Q-learning path with 100,000 iterations	Dijkstra path
241	'241 - Normal', '242 - Normal', '243 - Normal', '280 - Normal', '281 - Normal', '92 - Normal', '91 - Hacker'	241 - Normal', '242 - Normal', '243 - Normal', '280 - Normal', '281 - Normal', '92 - Normal', '91 - Hacker'
37	'37 – Target User', '36 – Target User', '35 – Target User', '34 – Target User', '25 – Target User', '26 – Target User', '27 – Target User', '28 – Target User', '209 – Cyber Police', '91 - Hacker'	'37 – Target User', '293 - Normal', '292 - Normal', '291 - Normal', '79 – Target User', '80 - Normal', '208 – Cyber Police', '209 – Cyber Police', '91 - Hacker'
187	187 – Infected Devices', '186 – Infected Devices', '0 - Normal', '1 - Normal', '2 - Normal', '3 - Normal', '4 - Normal', '79 – Target User', '80 - Normal', '208 – Cyber Police', '209 – Cyber Police', '91 - Hacker'	'187 – Infected Devices', '186 – Infected Devices', '0 - Normal', '128 – Infected Devices', '127 – Infected Devices', '126 – Infected Devices', '125 – Infected Devices', '124 – Infected Devices', '91 - Hacker'
40	'40 – Target User', '39 – Target User', '38 – Target User', '37 – Target User', '36 – Target User', '35 – Target User', '34 – Target User', '25 – Target User', '26 – Target User', '27 – Target User', '28 – Target User', '209 – Cyber Police', '91 - Hacker'	'40 – Target User', '288 - Normal', '289 - Normal', '206 – Cyber Police', '207 – Cyber Police', '208 – Cyber Police', '209 – Cyber Police', '91 - Hacker'
183	'183 – Infected Devices', '184 – Infected Devices', '262 - Normal', '247 - Normal', '246 - Normal', '245 - Normal', '244 - Normal', '243 - Normal', '280 - Normal', '281 - Normal', '92 - Normal', '91 - Hacker'	'183 – Infected Devices', '184 - Infected Devices', '155 - Infected Devices', '154 - Infected Devices', '243 - Normal', '280 - Normal', '281 - Normal', '92 - Normal', '91 - Hacker'

Conclusion:

In summary, the project concludes with the identification of optimal paths that strategically prioritize target users while penalizing the traversal through infected nodes. This approach guides the graph to bypass nodes systematically in accordance with specified rules until the penalized nodes are surpassed. It is acknowledged that certain nodes may not afford opportunities for deviation from the shortest path. To address this, the implementation of the SARSA algorithm is proposed to further minimize deviations and enhance the efficiency of pathfinding in the given context.

SARSA, an acronym for State-Action-Reward-State-Action, represents a reinforcement learning paradigm where the agent refines its policy through continuous updates to Q-values. This updating process incorporates information about the current state, the action taken, the immediate reward, the subsequent state, and the ensuing action. By applying the SARSA algorithm, deviations

from the shortest path can be minimized, refining the efficiency of the pathfinding process. This integration introduces a more resilient and adaptive approach, enriching the project's optimization and bolstering its efficacy in navigating intricate environments.

References:

- GeeksforGeeks. (2023, December 3). "ML | Reinforcement Learning Algorithm (Python Implementation using Q-learning)." Retrieved from <https://www.geeksforgeeks.org/ml-reinforcement-learning-algorithm-python-implementation-using-q-learning/>
- Baeldung. (2023, December). "DFS vs BFS vs Dijkstra's Algorithm." Retrieved from <https://www.baeldung.com/cs/dfs-vs-bfs-vs-dijkstra>
- Guo, S., & Cao, J. (2021). Reactive Power Optimization for Voltage Stability in Energy Internet Based on Graph Convolutional Networks and Deep Q-learning. In 2021 4th IEEE International Conference on Industrial Cyber-Physical Systems (ICPS) (pp. 511-516). Victoria, BC, Canada. doi: 10.1109/ICPS49255.2021.9468118.
- Soni, H., Vyas, R., & Hiran, K. K. (2022). Self-Autonomous Car Simulation Using Deep Q-Learning Algorithm. In 2022 International Conference on Trends in Quantum Computing and Emerging Business Technologies (TQCEBT) (pp. 1-4). Pune, India. doi: 10.1109/TQCEBT54229.2022.10041614.